TOKENTA VERIFICATION ENGINE

Architectur

Document type: Research / Engineering execution plan (no implementation)

Date:

TABLE OF CONTENTS

1. Executive Summary

2. Strat

1. EXECUTIVE SUMMARY

Tokenta's core product is not an API key marketplace UI. The traded commodity is:

"Trusted API Asset State"

The value chain:

Raw API Key -> Verification Pipeline -> Verified API Asset -> Tradable Asset

The Verification Engine is Tokenta's primary technical moat. The Terminal (BUY/SELL,

F1-F7 works

This document defines:

- How eac

No code implementation is included. This is the execution blueprint for engineering.

2. STRATEGIC POSITION: VERIFICATION AS MOAT

Competitive modules ranked by defensibility:

Module	Barrier	Notes

Long-term positioning:

Tokenta = API Asset Verification Network

Marketp

3. CURRENT FRONTEND SURFACE (TERMINAL UI AUDIT)

Based on: Tokenta Terminal Design Proposal (two-mode BUY/SELL architecture).

3.1 Navigation - F2 Verification Workspace

Backend expectation:

- Globa

3.2 SELL - List on Marketplace (P2P Listing)

Spec alignment (product doc):

- Form:

3.3 SELL - Tokenta Buyback

3.4 BUY - Pre-Purchase Verification

3.5 Market Listings - Verification Display

3.6 Orders - Verification Trail

3.7 Status Strip (Network Pulse)

---

4. END-TO-END USER PIPELINES (UI -> API)

4.1 Pipeline A - Seller Lists API (P2P) - PRIMARY ONBOARDING FLOW

```
  [UI] SELL -> ListApiModal -> LIST API ? START VERIFY
                                                      |
```

4.2 Pipeline B - Tokenta Buyback Sale

```
  [UI] SELL -> TOKENTA BUYBACK -> SELL TO TOKENTA -> quantity + key
                                                      |
```

4.3 Pipeline C - Buyer Pre-Purchase Re-Verification (P2P)

```
  [UI] F2 BUY context or Order detail -> REQUEST NEW VERIFICATION
                                                      |
```

4.4 Pipeline D - Order-Bound Delivery Verification (Key Rotation)

```
  [UI] F4 Chat / rotation guide -> seller submits new key
```

4.5 Pipeline E - Tokenta Direct Inventory (Platform Seller)

Not user-submitted keys in Terminal. Internal ops console provisions inventory.

5. SYSTEM ARCHITECTURE OVERVIEW

Design principles:

6. ASYNC VERIFICATION ARCHITECTURE

WHY ASYNC (mandatory):
  - Provider latency: 200ms-30s per probe; multiple probes per job
  - Rate li

ANTI-PATTERN (forbidden):
  POST /verify -> server blocks 30s -> returns PASS
  This freezes Terminal UI and violates trust UX (no visible pipeline).
CORRECT PATTERN:
  POST /jobs -> 202 Accepted { jobId }
  Client ->

Job SLA tiers:

Tier	Target duration	Context

7. REST API SPECIFICATION

Base URL: https://api.tokenta.network/v1
Auth: Beare

7.1 Start V

Request:
{

Response 202:
{

Errors:
  400 INVA

7.2 Get Job

Response 200:

{

Note: Response NEVER includes apiKey or full provider response bodies.

7.3 Cancel

Response 200: { "status": "CANCELLED" }

7.4 List Gl

Query: asset, result, method, since

Response 200:

{

7.5 Pipelin

Response 200:

{

Response 200:

Request: { "verificationJobId": "verif_01HXYZ", "proofId": "proof_abc" }

Fails 409 if job not PASSED or proof expired (TTL).

Request:

Response 201: { "orderId", "escrowId", "rotationDeadline" }

Body: { context: "INVENTORY_INGEST", provider, encryptedKeyRef, ... }

8. REAL-TIME EVENT STREAM (SSE / WebSocket)

Primary: Server-Sent Events (SSE) - simpler, Terminal-friendly, unidirectional
GET /v1/verification/jobs/{jobId}/events

Event format (JSON in data:):
  event: step_started

  event: step_completed

7.6 Get Pro

{

7.7 Attach

Response 20

7.8 Tokenta

{

7.9 Interna

Headers: Au

  data: {"j

```
  event: job_progress
  event: job_completed
  event: job_failed

UI mapping (seller onboarding - matches product spec):
  [x] Job Created          <- event: job_created (immediate after POST)

Buyback 3-step UI:
  Test 1/3 API connectivity  -> CONNECTIVITY

WebSocket alternative (optional for F2 global feed):
  WS /v1/verification/stream

Terminal F2 table appends rows on network.feed events (anonymized job refs).
Reconnect policy:
```

9. VERIFICATION WORKER PIPELINE (EXECUTION LAYERS)

```
Worker consumes: verification.jobs queue message { jobId }
Stage order (configurable per context):
  STAGE 0 - job_created
  STAGE 1 - CONNECTIVITY (Layer 1)
  STAGE 2 - QUOTA (Layer 2) - HARDEST
  STAGE 3 - RATE_LIMIT (Layer 2b)
  STAGE 4 - ACCESS_METHOD (Layer 3)
  STAGE 5 - RISK_ANALYSIS (Layer 4)
  STAGE 6 - FINALIZE

Failure handling:
```

- Retryab

10. PROVIDER ADAPTER LAYER

Directory structure (implementation reference):
  /providers
    /openai

Interface (conceptual):
  interface ProviderVerifier {
    provide

OpenAI quota strategies (multi-pronged):
- usage A

Anthropic:
- credits

Google Gemini:
- cloud p

Each adapter isolated for:
- Legal/T

This layer is the highest barrier to entry for competitors.

11. ACCESS METHOD VERIFICATION MATRIX

  Access Method Verification approach

Maps to trading-types.ts AccessMethod and VerificationMethod enums.
Frontend VerificationModeBanner maps to worker routing:
DIRECT_KE

12. RISK ANALYSIS ENGINE

Inputs:
- keyFing

Checks:

1. Shared key detection
2. Blacklist
3. Abnormal usage velocity
4. Reseller / gray market proxy
5. Revocation risk score

Output:

Feeds:

13. VERIFICATION PROOF & HASH (ON-CHAIN ANCHOR)

Canonical payload (sorted JSON):

```
{
```

proofHash = sha256(canonical_payload_bytes)

On-chain (optional Phase 2):

Order detail "DOWNLOAD HASH" -> GET /proofs/{id}

14. DATA MODEL & PERSISTENCE

Table: verification_jobs

Table: verification_job_steps

step_id

Table: verification_proofs

proof_id

Table: key_fingerprints

fingerpri

Secrets:

encrypted

Queue:

Redis Str

15. SECURITY: API KEY LIFECYCLE

INGEST (POST /jobs)

- TLS 1

WORKER

- Decry

AFTER VERIFICATION

- Raw k

DISPLAY

- Termi

RE-VERIFY

- User

16. CONTINUOUS VERIFICATION NETWORK (PHASE 2+)

Beyond one-shot jobs:

Scheduled heartbeat jobs per active listing / inventory SKU

Intervals

Worker lite pipeline: CONNECTIVITY + QUOTA quick probe

State transitions:

LIVE ->

Market impact:

Auto-de

This enables "real-time verified API market" positioning.

17. SERVICE TOPLOGY & DEPLOYMENT

Recommended initial stack:

- API Gateway: Kong / AWS API Gateway / Cloudflare
- Verific

Network isolation:

- Workers

Multi-region (later):

- Provide

18. OBSERVABILITY & SLOs

Metrics (Prometheus / Datadog):

- verific

SLOs:

- 99% of

Alerts:

- queue_d

F2 UI "Pipeline Health" = direct query of these metrics.

19. IMPLEMENTATION PHASES & MILESTONES

Phase 0 - Foundation (Weeks 1-2)

- Job mod

Phase 1 - Seller Onboarding (Weeks 3-4)

- Full pi

Phase 2 - Buyback & Rotation (Weeks 5-6)

- TOKENTA

Phase 3 - Buyer Flows (Weeks 7-8)

- PRE_PUR

Phase 4 - Continuous LIVE (Weeks 9-12)

- Heartbe

Phase 5 - Advanced proofs (Future)

- ZK/TEE

20. APPENDIX A: UI TRIGGER -> API MATRIX

UI Location	Action	API Endpoint

---

21. APPENDIX B: VERIFICATION JOB STATE MACHINE

PENDING -> RUNNING -> PASSED

Step states: PENDING -> RUNNING -> PASS | FAIL | SKIP
PASSED requires: all required stages PASS (or WARN accepted per policy)

22. APPENDIX C: EVENT TYPE CATALOG

job_created

step_star

Each event includes: jobId, timestamp, traceId (for support correlation)

END OF DOCUMENT

Tokenta Verification Engine Spec v0.1 - For internal engineering use.